

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Jesvita Dias(1BM23CS126)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Dec-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Jesvita Dias (1BM23CS126)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Sreevidya B S Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	20-8-2025	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	04-08
2	28-8-2025	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	08-12
3	3-9-2025	Implement A* search algorithm	13-16
4	10-9-2025	Implement Hill Climbing search algorithm to solve N-Queens problem	16-19
5	17-9-2025	Simulated Annealing to Solve 8-Queens problem	19-21
6	24-9-2025	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	21-23
7	8-10-2025	Implement unification in first order logic	23-25
8	15-10-2025	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	25-28
9	29-10-2025	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	28-31
10	12-11-2025	Implement Alpha-Beta Pruning.	31-33

Github Link:

<https://github.com/JesvitaDias/AI-Lab-5-SEM>

Program 1

Implement Tic – Tac – Toe Game

Implement vacuum cleaner agent

Algorithm:

```
Define Human = 'X', AI = 'O'
// Create 3x3 grid with 3 rows and 3 columns initialize all cells to Empty
board[3][3] = Empty

function findwinningmove(board, sym)
    lines = [(0,0),(0,1),(0,2)], [(1,0),(1,1),(1,2)], [(2,0),(2,1),(2,2)],
            [(0,0),(1,0),(2,0)], [(0,1),(1,1),(2,1)], [(0,2),(1,2),(2,2)],
            [(0,0),(0,1),(0,2)], [(1,0),(1,1),(1,2)], [(2,0),(2,1),(2,2)]
    for line in lines:
        if all board[r][c] == sym for (r,c) in line:
            return true
    return false

function findemptycells(board)
    list = []
    for r in range(0..2)
        for c in range(0..2)
            if board[r][c] == Empty
                list.append((r,c))
    return list

function choosmove(board)
    move = findwinningmove(board, AI)
    if move
        return move

function findminningmove(board, sym)
    for (r,c) in emptylist
        place sym in board[r][c]
        if iswin(board, sym)
            undo(r,c)
            return (r,c)
        undo(r,c)
    return null
```

```
function choosmove(board)
    move = findwinningmove(board, AI)
    if move != null return move
    move = findwinningmove(board, HUMAN)
    if move != null return move
    center = board[1][1]
    if center == Empty return center
    corners = [(0,0), (0,2), (2,0), (2,2)]
    for (r,c) in corners:
        if board[r][c] == Empty
            return (r,c)
    sides = [(0,1), (1,0), (1,2), (2,1)]
    for (r,c) in sides:
        if board[r][c] == Empty
            return (r,c)
    print("No empty cell")
    return null

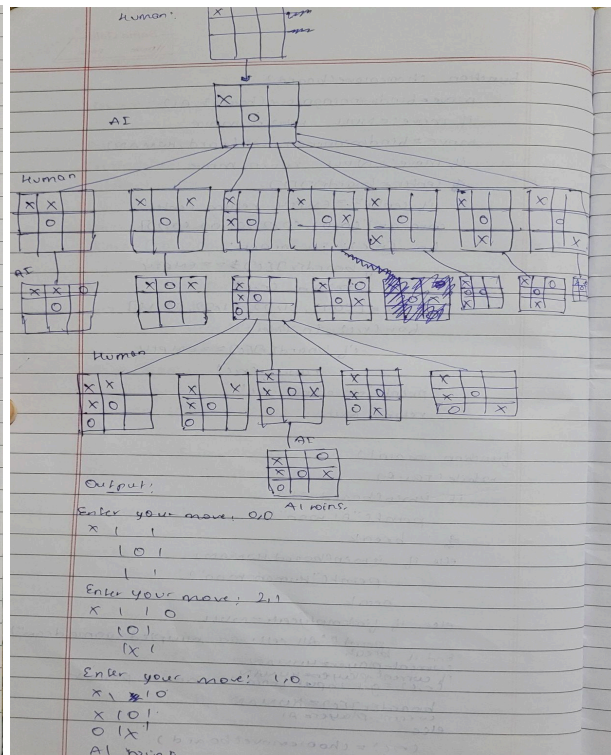
function main()
    while (True)
        if iswin(board, AI)
            print("AI won")
            break
        else if iswin(board, HUMAN)
            print("Human won")
            break
        else if findemptycells == null
            print("All cells are occupied, game draw")
            break
        currentPlayer = HUMAN
        if currentPlayer == HUMAN
            (r,c) = get input from human to place x
            board[r][c] = HUMAN
            currentPlayer = AI
        else
            (r,c) = choosmove(board)
            board[r][c] = AI
            currentPlayer = HUMAN
```

```

function choosemove(board)
    move = findwinningmove(board, AI)
    if move != null return move
    move = findwinningmove(board, HUMAN)
    if move != null return move
    center = board[1][1]
    if center == EMPTY return center
    corners = [(0,0), (0,2), (2,0), (2,2)]
    for (r,c) in corners:
        if board[r][c] == EMPTY
            return (r,c)
    sides = [(0,1), (1,0), (1,2), (2,1)]
    for (r,c) in sides:
        if board[r][c] == EMPTY
            return (r,c)
    print("no empty cell")
    return null

function main()
    while (true)
        iswin(board, AI)
        print("AI won")
        break
    else if iswin(board, HUMAN)
        print("Human won")
        break
    else if findemptycells == null
        print("All cells are occupied, game draw")
        break
    end if
    current_player = HUMAN
    if current_player == HUMAN
        (r,c) = get input from human to place x
        board[r][c] = HUMAN
        current_player = AI
    else
        (r,c) = choosemove(board)
        board[r][c] = AI
        current_player = HUMAN

```



```

EMPTY = " "
board = [[EMPTY for _ in range(3)] for _ in range(3)]
def print_board(board):
    for row in board:
        print("|".join(row))
        print("-" * 5)
def isWin(board, sym):
    lines = [ [(0,0),(0,1),(0,2)], [(1,0),(1,1),(1,2)], [(2,0),(2,1),(2,2)], [(0,0),(1,0),(2,0)],
[(0,1),(1,1),(2,1)], [(0,2),(1,2),(2,2)], [(0,0),(1,1),(2,2)], [(0,2),(1,1),(2,0)]]
    for line in lines:
        if all(board[r][c] == sym for (r,c) in line):
            return True
    return False
def emptyCells(board):
    return [(r,c) for r in range(3) for c in range(3) if board[r][c] == EMPTY]
def findWinningMove(board, sym):
    for (r,c) in emptyCells(board):
        board[r][c] = sym
        if isWin(board, sym):
            board[r][c] = EMPTY
            return (r,c)
        board[r][c] = EMPTY
    return None
def chooseMove(board):
    move = findWinningMove(board, AI)
    if move: return move
    move = findWinningMove(board, HUMAN)
    if move: return move
    if board[1][1] == EMPTY:
        return (1,1)
    for (r,c) in [(0,0),(0,2),(2,0),(2,2)]:
        if board[r][c] == EMPTY:
            return (r,c)
    for (r,c) in [(0,1),(1,0),(1,2),(2,1)]:
        if board[r][c] == EMPTY:
            return (r,c)
    return None
currentPlayer = HUMAN
while True:
    print_board(board)
    if isWin(board, AI):
        print("AI (O) wins!")
        break
    if isWin(board, HUMAN):
        print("You (X) win!")
        break
    if not emptyCells(board):

```

```

    print("It's a draw!")
    break
if currentPlayer == HUMAN:
    move=input("Enter your move as row,col (0-2,0-2): ")
    r,c=map(int, move.split(","))
    if board[r][c]==EMPTY:
        board[r][c]=HUMAN
        currentPlayer=AI
    else:
        print("Cell already taken! Try again.")
else:
    r,c=chooseMove(board)
    board[r][c]=AI
    currentPlayer=HUMAN

```

Vaccum cleaner:

```

a=input("Enter state of room A")
b=input("Enter state of room B")
state = {"A": "Dirty", "B": "Dirty"}
current_room = input("Enter current room")
print("Initial State:")
print(state)
while state["A"] == "Dirty" or state["B"] == "Dirty":
    print(f'Vacuum is in Room {current_room}')
    if state[current_room] == "Dirty":
        print(f'Cleaning Room {current_room}...')
        state[current_room] = "Clean"
    else:
        print(f'Room {current_room} is already clean.')
        current_room = "B" if current_room == "A" else "A"
    print(f'Moving to Room {current_room}...\n')
print("Final State:",state)
print("Both rooms are clean. Vacuum stopped.")

```

Output:


```

105:
Step 1: start
Step 2: input (start, goal)
Step 3: DFS (start, goal)
    depth = 0
    while True:
        result = DLS (start, goal, depth)
        if result != 'cutoff':
            return result
        depth += 1
    DLS (start, goal, limit)
    RDFS (start, goal, limit, 1)
    RDFS (state, goal, limit, path)
    if state == goal:
        return path
    if depth == 0:
        return 'cutoff'
    cutoff_heuristic = False
    for neighbour in expand(state):
        if neighbour not in path:
            result = RDFS (neighbour, goal, limit-1, path+neighbour)
            if result == 'cutoff':
                cutoff_heuristic = True
            else if result != 'failure':
                return result
    if cutoff_heuristic == True:
        return 'cutoff'
    else:
        return 'failure'

```

Code:

Depth First Search:

```

from copy import deepcopy
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
def swap(state, i1, j1, i2, j2):
    copy = deepcopy(state)
    copy[i1][j1], copy[i2][j2] = copy[i2][j2], copy[i1][j1]
    return copy
def print_state(state):
    for row in state:
        print(row)
def state_to_string(state):
    return ".join(str(num) for row in state for num in row)
def succ(state):
    i, j = find_blank(state)
    s = []
    if i > 0:
        s.append(swap(state, i, j, i-1, j))
    if i < 2:
        s.append(swap(state, i, j, i+1, j))

```

```

    if j > 0:
        s.append(swap(state, i, j, i, j-1))
    if j < 2:
        s.append(swap(state, i, j, i, j+1))
    return s
def dfs(state, goal, visited):
    print_state(state)
    if state == goal:
        return True
    state_str = state_to_string(state)
    visited.add(state_str)
    for s in succ(state):
        if state_to_string(s) not in visited:
            if dfs(s, goal, visited):
                return True
    return False
start = []
for _ in range(3):
    start.append(list(map(int, input().split())))
goal = []
for _ in range(3):
    goal.append(list(map(int, input().split())))
visited = set()
if dfs(start, goal, visited):
    print("Solution found")
else:
    print("No solution")

```

Iterative Deepening Search:

```

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                return i,j
def swap(state,i1,j1,i2,j2):
    copy=deepcopy(state)
    copy[i1][j1],copy[i2][j2]=copy[i2][j2],copy[i1][j1]
    return copy
def print_state(state):
    for row in state:
        print(row)
def state_to_string(state):
    return ".join(str(num) for row in state for num in row)
def succ(state):
    i,j=find_blank(state)
    successors=[]

```

```

if i>0:
    successors.append(swap(state,i,j,i-1,j))
if i<2:
    successors.append(swap(state,i,j,i+1,j))
if j>0:
    successors.append(swap(state,i,j,i,j-1))
if j<2:
    successors.append(swap(state,i,j,i,j+1))
return successors
def dls(state,goal,limit,visited):
    print_state(state)
    if state==goal:
        return True
    if limit==0:
        return False
    state_str=state_to_string(state)
    visited.add(state_str)
    for s in succ(state):
        s_str=state_to_string(s)
        if s_str not in visited:
            if dls(s,goal,limit-1,visited):
                return True
    return False
def ids(start,goal):
    depth=0
    while True:
        visited=set()
        if dls(start,goal,depth,visited):
            print("Solution found")
            return True
        depth+=1
        if depth>20:
            print("No solution")
            return False
start=[]
for _ in range(3):
    row=list(map(int,input().split()))
    start.append(row)
goal=[]
for _ in range(3):
    row=list(map(int,input().split()))
    goal.append(row)
ids(start,goal)

```

Depth First Search:

```
===== RESTART: C:\Us
enter input
1 2 3
4 5 6
7 8 0
1 2 3
4 5 6
7 0 8
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
[1, 2, 3]
[4, 5, 0]
[7, 8, 6]
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
Solution found
..
```

Iterative Deepening Search:

```
===== RESTART: C:\Users\Jesvita\On
1 2 3
4 0 5
6 7 8
1 2 3
4 5 0
6 7 8
[1, 2, 3]
[4, 0, 5]
[6, 7, 8]
[1, 2, 3]
[4, 0, 5]
[6, 7, 8]
[1, 0, 3]
[4, 2, 5]
[6, 7, 8]
[1, 2, 3]
[4, 7, 5]
[6, 0, 8]
[1, 2, 3]
[0, 4, 5]
[6, 7, 8]
[1, 2, 3]
[4, 5, 0]
[6, 7, 8]
Solution found
.\|
```

Program 3

Implement A* search algorithm

A* Heuristic Search - 8 puzzle problem

```

function AStar(start, goal, heuristicType):
    Open ← priority queue ordered by f(n)
    closed ← empty set
    g(start) = 0
    h(start) = calculateHeuristic(start, goal, heuristicType)
    f(start) = g(start) + h(start)
    add start to Open
    while Open is not empty:
        current ← node in Open with lowest f(n)
        if current == goal:
            return "Searched"
        remove current from Open
        add current to closed
        for each neighbour in expand(current):
            if neighbour in closed:
                continue
            g(neighbour) = g(current) + 1
            h(neighbour) = calculateHeuristic(neighbour, goal, heuristicType)
            f(neighbour) = g(neighbour) + h(neighbour)
            if neighbour not in Open:
                add neighbour to Open
            else if f(neighbour) is better:
                update Open with better f(neighbour)

function calculateHeuristic(state, goal, heuristicType):
    if heuristicType == "manhattan":
        return countManhattan(state, goal)
    else if heuristicType == "manhattan":
        return sumOfManhattan(state, goal)
    
```

Unplaced Tiles

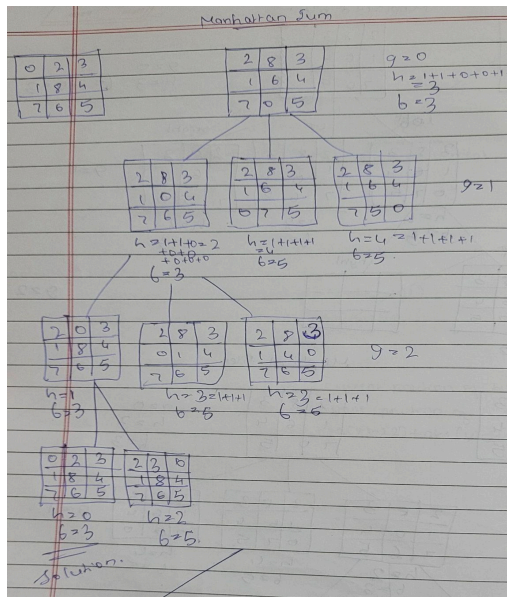
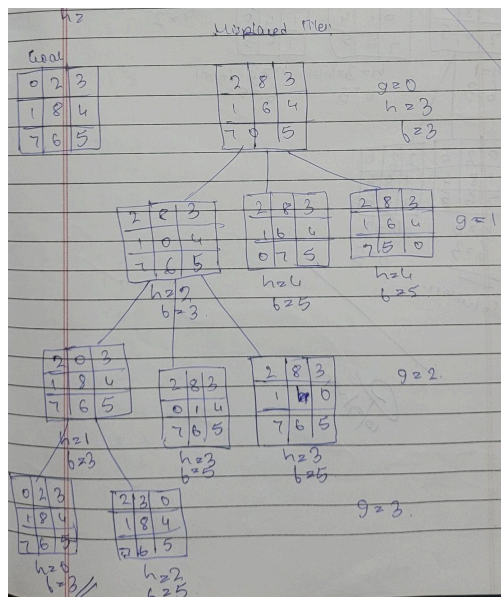
```

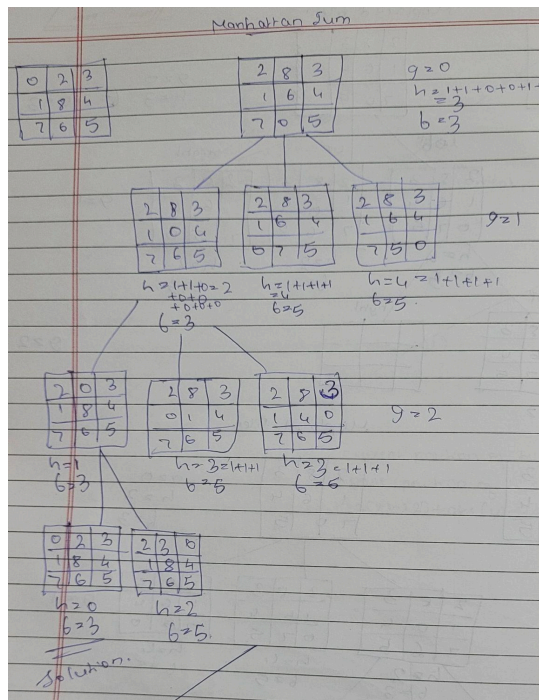
function countUnplaced(state, goal)
    count = 0
    for tile in state:
        if tile != blank and tile not in current position:
            count += 1
    return count
    
```

Manhattan sum

```

function sumManhattan(state, goal)
    distance = 0
    for tile in state:
        if tile != blank:
            (x1, y1) ← current position of tile
            (x2, y2) ← goal position of tile
            distance += abs(x1 - x2) + abs(y1 - y2)
    return distance
    
```





Code:

```
from copy import deepcopy
```

```
import heapq
```

```
def find_blank(state):
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] == 0:
```

```
                return i, j
```

```
def swap(state, i1, j1, i2, j2):
```

```
    copy = deepcopy(state)
```

```
    copy[i1][j1], copy[i2][j2] = copy[i2][j2], copy[i1][j1]
```

```
    return copy
```

```
def print_state(state):
```

```
    for row in state:
```

```
        print(row)
```

```
def state_to_string(state):
```

```
    return ".join(str(num) for row in state for num in row)
```

```
def succ(state):
```

```
    i, j = find_blank(state)
```

```
    s = []
```

```
    if i > 0:
```

```
        s.append(swap(state, i, j, i - 1, j))
```

```
    if i < 2:
```

```
        s.append(swap(state, i, j, i + 1, j))
```

```
    if j > 0:
```

```
        s.append(swap(state, i, j, i, j - 1))
```

```

    if j < 2:
        s.append(swap(state, i, j, i, j + 1))
    return s
def misplaced_tiles(state, goal):
    return sum(1 for i in range(3) for j in range(3) if state[i][j] != 0 and state[i][j]
!= goal[i][j])
def manhattan(state, goal):
    pos = {}
    for i in range(3):
        for j in range(3):
            pos[goal[i][j]] = (i, j)
    d = 0
    for i in range(3):
        for j in range(3):
            num = state[i][j]
            if num != 0:
                gi, gj = pos[num]
                d += abs(i - gi) + abs(j - gj)
    return d
def astar(start, goal, heuristic_type):
    open_heap = []
    visited = set()
    g = {state_to_string(start): 0}
    if heuristic_type == "misplaced":
        h = misplaced_tiles(start, goal)
    else:
        h = manhattan(start, goal)
    heapq.heappush(open_heap, (h, start))
    while open_heap:
        f, state = heapq.heappop(open_heap)
        print_state(state)
        if state == goal:
            print("Solution found")
            return True
        state_str = state_to_string(state)
        if state_str in visited:
            continue
        visited.add(state_str)
        for s in succ(state):
            s_str = state_to_string(s)
            tentative_g = g[state_str] + 1
            if s_str not in g or tentative_g < g[s_str]:
                g[s_str] = tentative_g
                if heuristic_type == "misplaced":
                    h = misplaced_tiles(s, goal)
                else:
                    h = manhattan(s, goal)

```

```

        heapq.heappush(open_heap, (tentative_g + h, s))
    print("No solution")
    return False
start = []
for _ in range(3):
    start.append(list(map(int, input().split())))
goal = []
for _ in range(3):
    goal.append(list(map(int, input().split())))
choice = input("Enter heuristic (misplaced/manhattan): ").strip().lower()
astar(start, goal, choice)

```

Output:

```

1 2 3
4 5 0
6 7 8
1 2 3
4 0 5
6 8 7
Enter heuristic (misplaced/manhattan): misplaced
[1, 2, 3]
[4, 5, 0]
[6, 7, 8]
No solution

===== RESTART: C:\Users\Jesvita\OneDrive\Desktop\Co
1 2 3
4 5 0
6 8 7
1 2 3
4 8 5
6 7 0
Enter heuristic (misplaced/manhattan): manhattan
[1, 2, 3]
[4, 5, 0]
[6, 8, 7]
[1, 2, 3]
[4, 0, 5]
[6, 8, 7]
[1, 2, 3]
[4, 8, 5]
[6, 0, 7]
[1, 2, 3]
[4, 8, 5]
[6, 7, 0]
Solution found

```

Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem

```

function HillClimbing(N):
    state ← generateStateRandom(N)
    loop:
        currentConflict ← heuristic(state)
        bestNeighbour ← state
        bestConflict ← currentConflict
        for each column c from 1 to N:
            for each row r from 1 to N:
                if state[c] ≠ r then
                    neighbour ← copy(state)
                    neighbour[c] ← r
                    conflict ← heuristic(neighbour)

```

if conflict < bestconflict then
 bestneighbour ← neighbour
 bestconflict ← conflict
 if bestconflict < currentconflict
 state ← bestneighbour
 alle
 return state
 if bestconflict = 0 then
 return bestneighbour

function heuristic(state):
 h ← 0
 for i from 0 to N-1:
 for j from i+1 to N-1:
 if state[i] = state[j]:
 h ← h+1
 if ^{abs} state[i] - state[j] = abs(i-j):
 h ← h+1
 return h

$x_0=3 \quad x_1=2 \quad x_2=2 \quad x_3=0$
 0 1 2 3

0			x_3
1		x_1	
2		x_2	
3	x_0		

$x_0=3 \quad x_1=2 \quad x_2=2 \quad x_3=0$ pairs (least)
 3 0 1 2 0 0
 1 3 2 0 0 1
 0 3 2 0 0 1
 0 1 2 3 0 6
 3 2 1 0 0 6
 3 0 2 1 0 1
 3 0 1 0 2 1

$x_0=3 \quad x_1=2 \quad x_2=2 \quad x_3=0$
 0 1 2 3

0			x_3
1		x_1	
2		x_2	
3	x_0		

$x_0=3 \quad x_1=2 \quad x_2=2 \quad x_3=0$ pairs (least)
 3 0 1 2 0 0
 1 3 2 0 0 1
 0 3 2 0 0 1
 0 1 2 3 0 6
 3 2 1 0 0 6
 3 0 2 1 0 1
 3 0 1 0 2 1

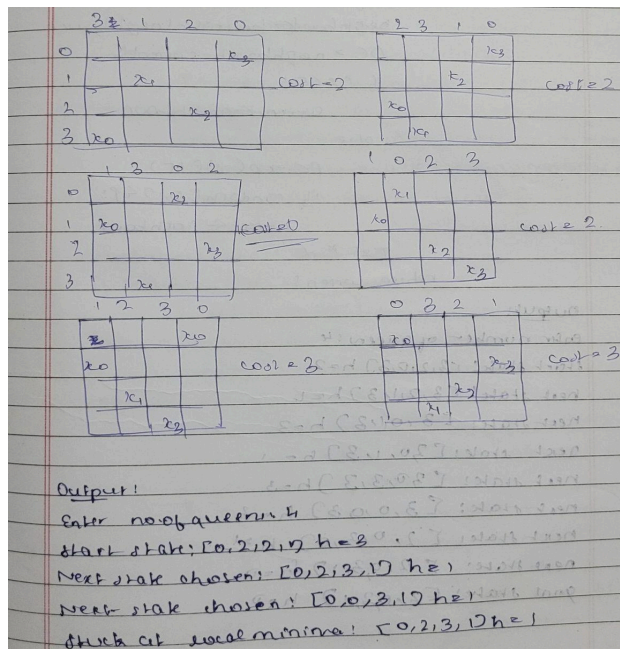
0 1 2 3
 0 x_3 x_1 x_2
 1 x_0 x_2 x_3
 2 x_1 x_0 x_2
 3 x_2 x_1 x_0

0 1 2 3
 0 x_1 x_2 x_3
 1 x_2 x_1 x_0
 2 x_3 x_0 x_1
 3 x_0 x_1 x_2

0 1 2 3
 0 x_1 x_2 x_3
 1 x_2 x_1 x_0
 2 x_3 x_0 x_1
 3 x_0 x_1 x_2

least: $x_0 x_1 : (3 2 0)$ cost = 1

1	3	2	0	pairs	least
3	1	2	0	$x_0 x_1$	2
2	3	1	0	$x_0 x_2$	2
0	3	2	1	$x_0 x_3$	4
1	2	3	0	$x_1 x_2$	4
1	0	2	3	$x_1 x_3$	2
1	3	0	2	$x_2 x_3$	0



Code:

```

import random
def heuristic(state):
    h=0
    n=len(state)
    for i in range(n):
        for j in range(i+1,n):
            if state[i]==state[j]:
                h+=1
            if abs(state[i]-state[j])==abs(i-j):
                h+=1
    return h
def hillclimbing(state):
    print("Start state:", state, "h =", heuristic(state))
    while True:
        currenth=heuristic(state)
        if currenth==0:
            print("Reached goal state:", state, "h =", currenth)
            return state
        neighbors=[]
        n=len(state)
        for c in range(n):
            for r in range(n):
                if r!=state[c]:
                    newstate=state.copy()
                    newstate[c]=r
                    neighbors.append(newstate)
        bestneighbor=min(neighbors, key=heuristic)
        besth=heuristic(bestneighbor)
  
```

```

print("Next state chosen:", bestneighbor, "h =", besth)
if besth >= currenth:
    print("Stuck at local minimum:", state, "h =", currenth)
    return state
state = bestneighbor
n = int(input("Enter number of queens: "))
startstate = [random.randint(0, n-1) for _ in range(n)]
solution = hillclimbing(startstate)

```

Output:

```

----- RESTART: C:\Users\vesvica\OneDrive\Desktop\College\pr
Enter number of queens: 8
Start state: [5, 1, 7, 5, 2, 3, 5, 4] h = 7
Next state chosen: [0, 1, 7, 5, 2, 3, 5, 4] h = 5
Next state chosen: [0, 1, 7, 4, 2, 3, 5, 4] h = 4
Next state chosen: [0, 1, 7, 4, 6, 3, 5, 4] h = 3
Next state chosen: [0, 1, 7, 4, 6, 3, 5, 2] h = 2
Next state chosen: [3, 1, 7, 4, 6, 3, 5, 2] h = 2
Stuck at local minimum: [0, 1, 7, 4, 6, 3, 5, 2] h = 2

```

Program 5

Simulated Annealing to Solve 8-Queens problem

```

function simulated(N, T, coolingrate)
    current ← GenerateRandomState(N)
    while T > 0:
        current ← heuristic(current)
        neighbour
        if currenth == 0:
            return current
        neighbour ← pickRandom(current)
        neighbour ← heuristic(neighbour)
        DE = neighbourh - currenth
        if DE ≤ 0:
            current ← neighbour
        else
            p ← exp(-DE/T)
            if random(0,1) ≤ p:
                current ← neighbour
        T = T * α
    return current

```

Code:

```
import random
import math
def heuristic(state):
    h = 0
    n=len(state)
    for i in range(n):
        for j in range(i+1,n):
            if state[i]==state[j]:
                h+=1
            if abs(state[i]-state[j])==abs(i-j):
                h+=1
    return h
def generaterandomstate(n):
    return [random.randint(0,n-1) for _ in range(n)]
def randomneighbor(state):
    n=len(state)
    neighbor=state.copy()
    c=random.randint(0,n-1)
    r=random.randint(0,n-1)
    while r==neighbor[c]:
        r=random.randint(0,n-1)
    neighbor[c]=r
    return neighbor
def simulatedannealing(n,T,coolingrate):
    current=generaterandomstate(n)
    print("start state:",current,"h=",heuristic(current))
    while T>0:
        currenth=heuristic(current)
        if currenth==0:
            print("reached goal state:",current,"h=",currenth)
            return current
        neighbor=randomneighbor(current)
        neighborh=heuristic(neighbor)
        delta=neighborh-currenth
        if delta<0:
            current=neighbor
        else:
            P=math.exp(-delta/T)
            if random.random()<P:
                current=neighbor
        print("next state chosen:",current,"h=",heuristic(current))
        T*=coolingrate
    print("final state:",current,"h=",heuristic(current))
    return current
n=int(input("enter number of queens: "))
```

T=100.0

coolingrate=0.99

solution=simulatedannealing(n, T, coolingrate)

```
===== RESTART: C:\Users\Jesvita\OneDri
enter number of queens: 4
start state: [1, 2, 3, 3] h= 4
next state chosen: [1, 2, 1, 3] h= 3
next state chosen: [1, 1, 1, 3] h= 4
next state chosen: [1, 1, 1, 2] h= 4
next state chosen: [1, 1, 1, 3] h= 4
next state chosen: [1, 0, 1, 3] h= 3
next state chosen: [1, 0, 1, 2] h= 5
next state chosen: [1, 0, 1, 3] h= 3
next state chosen: [1, 2, 1, 3] h= 3
next state chosen: [1, 2, 1, 0] h= 5
next state chosen: [1, 2, 0, 0] h= 3
next state chosen: [1, 2, 2, 0] h= 3
next state chosen: [1, 2, 1, 0] h= 5
next state chosen: [1, 2, 3, 0] h= 4
next state chosen: [2, 2, 3, 0] h= 3
next state chosen: [2, 2, 0, 0] h= 4
next state chosen: [2, 1, 0, 0] h= 4
next state chosen: [1, 1, 0, 0] h= 3
next state chosen: [1, 3, 0, 0] h= 1
next state chosen: [1, 3, 1, 0] h= 2
next state chosen: [1, 1, 1, 0] h= 4
next state chosen: [1, 2, 1, 0] h= 5
next state chosen: [1, 2, 1, 2] h= 5
next state chosen: [2, 2, 1, 2] h= 5
next state chosen: [2, 1, 1, 2] h= 4
next state chosen: [2, 3, 1, 2] h= 3
next state chosen: [2, 1, 1, 2] h= 4
next state chosen: [2, 1, 3, 2] h= 3
next state chosen: [2, 1, 1, 2] h= 4
next state chosen: [2, 1, 2, 2] h= 5
next state chosen: [2, 1, 1, 2] h= 4
next state chosen: [3, 1, 1, 2] h= 3
next state chosen: [3, 2, 1, 2] h= 5
next state chosen: [1, 2, 1, 2] h= 5
next state chosen: [1, 2, 0, 2] h= 2
next state chosen: [0, 2, 0, 2] h= 2
next state chosen: [0, 2, 2, 2] h= 4
next state chosen: [2, 2, 2, 2] h= 6
next state chosen: [2, 3, 2, 2] h= 5
next state chosen: [2, 3, 2, 1] h= 5
next state chosen: [2, 0, 2, 1] h= 2
next state chosen: [2, 0, 1, 1] h= 2
next state chosen: [0, 3, 2, 3] h= 5
next state chosen: [0, 3, 2, 2] h= 3
next state chosen: [0, 3, 1, 2] h= 1
next state chosen: [0, 3, 1, 3] h= 2
next state chosen: [0, 2, 1, 3] h= 2
next state chosen: [0, 1, 1, 3] h= 4
next state chosen: [0, 1, 0, 3] h= 5
next state chosen: [0, 1, 0, 2] h= 3
next state chosen: [1, 1, 0, 2] h= 2
next state chosen: [1, 0, 0, 2] h= 3
next state chosen: [1, 0, 0, 0] h= 4
next state chosen: [1, 1, 0, 0] h= 3
next state chosen: [1, 1, 3, 0] h= 2
next state chosen: [1, 1, 3, 1] h= 4
next state chosen: [3, 1, 3, 1] h= 2
next state chosen: [3, 1, 1, 1] h= 4
next state chosen: [3, 0, 1, 1] h= 3
next state chosen: [3, 1, 1, 1] h= 4
next state chosen: [3, 1, 3, 1] h= 2
next state chosen: [3, 1, 3, 0] h= 2
next state chosen: [3, 1, 3, 2] h= 2
next state chosen: [2, 1, 3, 2] h= 3
next state chosen: [2, 1, 0, 2] h= 4
next state chosen: [2, 1, 2, 2] h= 5
next state chosen: [2, 1, 2, 1] h= 5
next state chosen: [2, 1, 1, 1] h= 4
next state chosen: [2, 1, 1, 2] h= 4
next state chosen: [2, 3, 1, 2] h= 3
next state chosen: [2, 3, 1, 0] h= 2
next state chosen: [2, 3, 1, 1] h= 3
next state chosen: [1, 3, 1, 1] h= 4
next state chosen: [1, 1, 1, 1] h= 6
next state chosen: [1, 1, 1, 0] h= 4
next state chosen: [1, 2, 1, 0] h= 5
next state chosen: [1, 1, 1, 0] h= 4
next state chosen: [2, 1, 1, 0] h= 3
next state chosen: [1, 1, 1, 0] h= 4
next state chosen: [2, 1, 1, 1] h= 4
next state chosen: [2, 3, 1, 1] h= 3
next state chosen: [2, 0, 1, 1] h= 2
next state chosen: [2, 0, 3, 1] h= 0
reached goal state: [2, 0, 3, 1] h= 0
```

Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

```
ModelSatisfier(KB,  $\alpha$ ):
    symbols  $\leftarrow$  all symbols in KB and  $\alpha$ 
    return ModelSatisfier(KB,  $\alpha$ , symbols, 1)

ModelSatisfier(KB,  $\alpha$ , symbols, model):
    if symbols is empty:
        if ModelSatisfier(model, KB):
            return ModelSatisfier(model,  $\alpha$ )
        else:
            return true
    else:
        p  $\leftarrow$  first symbol in symbols
        rest  $\leftarrow$  symbols - p
        result1  $\leftarrow$  ModelSatisfier(KB,  $\alpha$ , rest, model  $\vee$  (p = true))
        result2  $\leftarrow$  ModelSatisfier(KB,  $\alpha$ , rest, model  $\vee$  (p = false))
        return result1 and result2
```

```
ModelSatisfier(model, sentence):
    if sentence is a single symbol:
        return model[sentence]
    else if sentence is  $\neg \phi$  then
        return not ModelSatisfier(model,  $\phi$ )
    else if sentence is  $\phi \wedge \psi$  then
        return ModelSatisfier(model,  $\phi$ ) AND
            ModelSatisfier(model,  $\psi$ )
    else if sentence is  $(\phi \vee \psi)$ :
        return ModelSatisfier(model,  $\phi$ ) OR
            ModelSatisfier(model,  $\psi$ )
    else if sentence is  $(\phi \rightarrow \psi)$ :
        return (not ModelSatisfier(model,  $\phi$ )) OR
            ModelSatisfier(model,  $\psi$ )
```

KB: $Q \rightarrow P$
 $P \rightarrow \neg Q$
 $Q \vee R$

$R \rightarrow P$
 $Q \rightarrow R$

Truth table:

Q	P	$Q \rightarrow P$	$P \rightarrow \neg Q$	$Q \vee R$	$R \rightarrow P$	$Q \rightarrow R$	KB
T	T	T	F	T	T	T	F
T	F	F	T	F	F	T	F
F	T	T	T	T	T	F	F
F	F	T	T	F	F	T	F
T	T	T	F	T	T	T	T
T	F	F	T	F	F	T	F
F	T	T	T	T	T	F	F
F	F	T	T	F	F	T	F

$\alpha \equiv R \rightarrow P$ $\beta \equiv Q \rightarrow R$

$KB \models \alpha$ $KB \models \beta$

$\therefore KB$ is true for $\alpha = R \rightarrow P$ and $\beta = Q \rightarrow R$ but α is false.

$\therefore$ α is true whenever KB is true.

α

$KB \models \alpha$

Code:

```
import itertools
def modelsatisfies(model,sentence):
    if isinstance(sentence,str):
        return model[sentence]
    elif sentence[0]=='¬':
        return not modelsatisfies(model,sentence[1])
    elif sentence[0]=='∧':
        return modelsatisfies(model,sentence[1]) and modelsatisfies(model,sentence[2])
    elif sentence[0]=='∨':
        return modelsatisfies(model,sentence[1]) or modelsatisfies(model,sentence[2])
    elif sentence[0]=='→':
        return (not modelsatisfies(model,sentence[1])) or modelsatisfies(model, sentence[2])
def getsymbols(sentence,symbols):
    if isinstance(sentence,str):
        symbols.add(sentence)
    elif isinstance(sentence, tuple):
        for part in sentence[1:]:
            getsymbols(part,symbols)
def printtruthtable(KB):
    symbols=set()
    for s in KB:
        getsymbols(s,symbols)
    symbols=sorted(list(symbols))
    header=" | ".join(symbols) + " || " + " | ".join([str(s) for s in KB])
    print(header)
    print("-" * len(header))
    for values in itertools.product([False, True],repeat=len(symbols)):
        model=dict(zip(symbols, values))
        row=" | ".join(['T' if model[s] else 'F' for s in symbols])
        row+=" || " + " | ".join(['T' if modelsatisfies(model, s) else 'F' for s in KB])
        print(row)
```

```
KB = [
    ('→', 'Q', 'P'),
    ('→', 'P', ('¬', 'Q')),
    ('∨', 'Q', 'R')
]
printtruthtable(KB)
```

Output:

```
===== RESTART: C:
P | Q | R || ('→', 'Q', 'P') | ('→', 'P', ('¬', 'Q')) | ('∨', 'Q', 'R')
-----
```

F	F	F	T	T	F
F	F	T	T	T	T
F	T	F	F	T	T
F	T	T	F	T	T
T	F	F	T	T	F
T	F	T	T	T	T
T	T	F	T	F	T
T	T	T	T	F	T

Program 7

Implement unification in first order logic.

```
Unify(x, y, θ):
    if θ = failure then
        return failure
    else if x == y then
        return θ
    else if isvariable(x)
        return unifyvar(x, y, θ)
    else if isvariable(y)
        return unifyvar(y, x, θ)
    else if iscompound(x) and iscompound(y)
        if operator(x) != operator(y) or arity(x) != arity(y)
            return failure
        else
            return unify(arguments(x), arguments(y), θ)
    else
        return failure
```

```
Unifyvar(var, x, θ):
    if var ∈ θ then
        return unify(θ[var], x, θ)
    else if x ∈ θ
        return unify(var, θ[x], θ)
    else if var occurs in x then
        return failure
    else
        extend θ with {var → x}
        return θ
```

Eats(x, Apple)
 Eats(Riya, y)
 unify(Eats(x, Apple), Eats(Riya, y))
 Operator $\rightarrow$ same Eats()
 arity $\rightarrow$ same (2)
 substitution: $\theta = \{x/Riya, y/Apple\}$
 Output:
 enter 1st expression: Father(x, ram)
 enter 2nd expression: Father(ranj, y)
 substitution: $\{x/'ranj', ram/'y'\}$

Code:

```

def isvariable(x):
    return isinstance(x, str) and x.islower()
def parseterm(expr):
    expr = expr.strip()
    if '(' in expr and expr.endswith(')'):
        func = expr[:expr.index('(')].strip()
        argsstr = expr[expr.index('(')+1:-1]
        args = [arg.strip() for arg in argsstr.split(',')]
        return (func, *args)
    else:
        return expr
def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    if x == y:
        return theta
    elif isvariable(x):
        return unifyvar(x, y, theta)
    elif isvariable(y):
        return unifyvar(y, x, theta)
    elif isinstance(x, tuple) and isinstance(y, tuple) and len(x) == len(y):
        for xi, yi in zip(x, y):
            theta = unify(xi, yi, theta)
            if theta is None:
                return None
        return theta
    else:
        return None
def unifyvar(var, x, theta):
    if var in theta:
        return unify(theta[var], x, theta)
    elif x in theta:
        return unify(var, theta[x], theta)
    else:
        theta[var] = x

```

```

    return theta
expr1=input("enter 1st expression: ")
expr2=input("enter 2nd expression: ")
term1=parseterm(expr1)
term2=parseterm(expr2)
result=unify(term1,term2)
if result is None:
    print("cant unify")
else:
    print("Substitution:",result)

```

Output:

```

=====
enter 1st expression: f(x,a)
enter 2nd expression: f(b,a)
Substitution: {'x': 'b'}

```

Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

As per data, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles all of which were sold to it by Robert who is an American.

Prove - 'Robert is criminal'

Knowledge Base:

- American(p) $\wedge$ Weapon(q) $\wedge$ Sell(p,q,r) $\wedge$ Hostile(r) $\Rightarrow$ Criminal(p)
- $\exists x$ Orns(A,x) $\wedge$ Missile(x)
- $\forall x$ Missile(x) $\wedge$ Orns(A,x) $\Rightarrow$ Sell(Robert,x,A)
- Missile(x) $\Rightarrow$ Weapon(x)
- $\forall x$ Enemy(x,America) $\Rightarrow$ Hostile(x)
- American(Robert)
- Enemy(A,America)

Existential instantiation $\rightarrow$ 2,1,1.

function: FOL-FC(KB, query):

```

new = {}
REPEAT
    for each rule in KB do:
        (P1^P2^...^Pn => Q) ← standardize-variables(rule)
        for each set of facts (p1,p2,...,pn) in KB:
            if unify(P1, p1, ..., Pn, p1, p2, ..., pn):
                if Q ≠ failure:
                    Q ← Apply(p, Q)
                    if Q is not in KB or new then:
                        add Q to new
                        Q ← unify(Q, query)
                    if Q ≠ fail:
                        return Q
                add all elements of new to KB:
            until new = {}
        return failure
    
```

Code:

```

def parse_predicate(expr):
    expr = expr.strip()
    if '=>' in expr:

```

```

    premise, conclusion = expr.split('=>')
    premises = [parse_predicate(p.strip()) for p in premise.split('&')]
    conclusion = parse_predicate(conclusion.strip())
    return (premises, conclusion)
if '(' in expr and expr.endswith(')'):
    pred_name = expr[:expr.index('(')]
    args = expr[expr.index('(')+1:-1].split(',')
    args = tuple(arg.strip() for arg in args)
    return (pred_name,) + args
return (expr,)
def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    if x == y:
        return theta
    elif isinstance(x, str) and x.islower():
        return unifyvar(x, y, theta)
    elif isinstance(y, str) and y.islower():
        return unifyvar(y, x, theta)
    elif isinstance(x, tuple) and isinstance(y, tuple) and len(x) == len(y):
        for xi, yi in zip(x, y):
            theta = unify(xi, yi, theta)
            if theta is None:
                return None
        return theta
    else:
        return None
def unifyvar(var, x, theta):
    if var in theta:
        return unify(theta[var], x, theta)
    elif isinstance(x, str) and x in theta:
        return unify(var, theta[x], theta)
    else:
        theta[var] = x
        return theta
def applysubstitution(expr, theta):
    if isinstance(expr, tuple):
        pred = expr[0]
        args = []
        for arg in expr[1:]:
            if arg in theta:
                args.append(theta[arg])
            else:
                args.append(arg)
        return (pred,) + tuple(args)
    return expr

```

```

def expr_to_str(expr):
    if isinstance(expr, tuple):
        if len(expr) == 1:
            return expr[0]
        return f"{expr[0]}({'', '.join(expr[1:])})"
    return str(expr)

def folfc(KB, query):
    inferred = set()
    added = True
    KB_parsed = []
    for rule in KB:
        if '=>' in rule:
            premises, conclusion = parse_predicate(rule)
            KB_parsed.append((premises, conclusion))
        else:
            KB_parsed.append(parse_predicate(rule))
    query_parsed = parse_predicate(query)
    facts = [fact for fact in KB_parsed if not isinstance(fact, tuple) or (isinstance(fact, tuple) and
not isinstance(fact[0], list))]
    rules = [fact for fact in KB_parsed if isinstance(fact, tuple) and isinstance(fact[0], list)]
    inferred_facts = set()
    while added:
        added = False
        for rule in rules:
            premises, conclusion = rule
            subs_list = [{}]
            for p in premises:
                new_subs_list = []
                for fact in facts + list(inferred_facts):
                    for theta in subs_list:
                        theta_new = unify(p, fact, dict(theta))
                        if theta_new is not None:
                            new_subs_list.append(theta_new)
                subs_list = new_subs_list
            for theta in subs_list:
                q = applysubstitution(conclusion, theta)
                q_str = expr_to_str(q)
                if q not in facts and q not in inferred_facts:
                    inferred_facts.add(q)
                    added = True
                    if unify(q, query_parsed) is not None:
                        print(f"Inferred: {q_str}")
                        return theta
            facts.extend(list(inferred_facts))
            inferred_facts.clear()
    return False

```

```

n = int(input("enter number of sentences in knowledge base: "))
KB = [input(f"enter sentence {i+1}: ") for i in range(n)]
query = input("enter query: ")
result = folfc(KB, query)
if result:
    print("Query entailed. Substitution:", result)
else:
    print("Query not entailed.")

```

Output:

```

>
=====
enter number of sentences in knowledge base: 4
enter sentence 1: Father(John,x)
enter sentence 2: Father(y,Bob)
enter sentence 3: Father(x,y) => Parent(x,y)
enter sentence 4: Parent(x, y) => Ancestor(x, y)
enter query: Ancestor(John, Bob)
Inferred: Ancestor(y, Bob)
Query entailed. Substitution: {'x': 'y', 'y': 'Bob'}
>

```

Program 9

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution

Algorithm Resolutionproof(KB, Q)

KB ← set of f.o.l statements {S₁, S₂, ..., S_n}

Q ← query to prove

convert all S_i ∈ KB and ¬Q to CNF

S ← KB ∪ {¬Q}

while True

 new ← ∅

 for each pair (C_i, C_j) ∈ S, where i < j do

 resolvent ← Resolve(C_i, C_j)

 if resolvent ≠ ∅

 return True

 new ← new ∪ resolvent

 end for

 if new ⊆ S

 return False

 S ← S ∪ new

End.

function Resolve(C_i, C_j)

R ← ∅

for each literal L ∈ C_i do

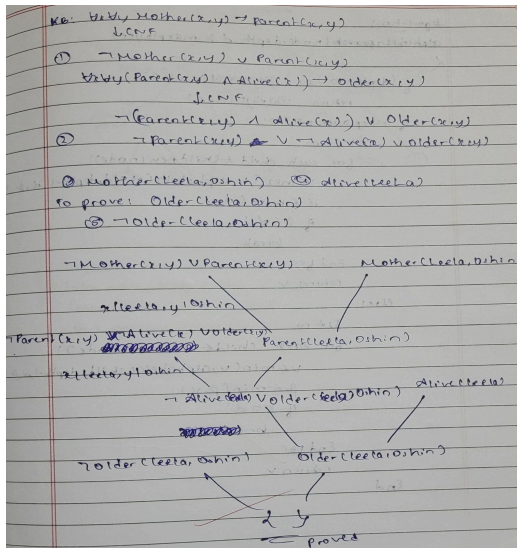
 if (¬L) ∈ C_j then

 R ← (C_i - {L}) ∪ (C_j - {¬L})

 end if

end for

return R



Code:

```
from itertools import combinations
def unify(x, y, subst):
    if subst is None:
        return None
    elif x == y:
        return subst
    elif isinstance(x, str) and x[0].islower(): # variable
        return unify_var(x, y, subst)
    elif isinstance(y, str) and y[0].islower(): # variable
        return unify_var(y, x, subst)
    elif isinstance(x, tuple) and isinstance(y, tuple) and x[0] == y[0] and len(x) == len(y):
        for a, b in zip(x[1:], y[1:]):
            subst = unify(a, b, subst)
            if subst is None:
                return None
        return subst
    else:
        return None
def unify_var(var, x, subst):
    if var in subst:
        return unify(subst[var], x, subst)
    elif x in subst:
        return unify(var, subst[x], subst)
    else:
        new_subst = subst.copy()
        new_subst[var] = x
        return new_subst
def substitute(literal, subst):
    """Apply substitution to a literal"""
    return (literal[0],) + tuple(subst.get(a, a) for a in literal[1:])
def negate(literal):
    if literal[0].startswith('~'):
        return (literal[0][1:],)+literal[1:]
```

```

else:
    return ('~'+literal[0,])+literal[1:]
def complementary(l1, l2):
    """Check if two literals are complementary"""
    return (l1[0] == '~' + l2[0]) or (l2[0] == '~' + l1[0])
def resolve(ci, cj):
    resolvents = []
    for li in ci:
        for lj in cj:
            if complementary(li, lj):
                subs = unify(li, negate(lj), {})
                if subs is not None:
                    new_ci = [substitute(l, subs) for l in ci if l != li]
                    new_cj = [substitute(l, subs) for l in cj if l != lj]
                    new_clause = list(set(new_ci + new_cj))
                    resolvents.append(tuple(sorted(new_clause)))
    return resolvents
def resolution(kb, query):
    clauses = [list(c) for c in kb] + [[negate(query)]]
    print("KNOWLEDGE BASE ")
    for i, c in enumerate(kb, 1):
        print(f"Clause {i}: {c}")
    print("\nQuery:", query)
    print("Initial Clauses:")
    for i, c in enumerate(clauses, 1):
        print(f"C{i}: {c}")
    print()
    step=1
    while True:
        new=[]
        for (ci, cj) in combinations(clauses, 2):
            resolvents=resolve(ci, cj)
            for res in resolvents:
                print(f"Step {step}: Resolving {ci} and {cj}")
                print(f" $\Rightarrow$ {res}")
                step+=1
                if not res:
                    print("\n Empty clause derived. QUERY IS ENTAILED.")
                    return True
            if list(res) not in clauses and list(res) not in new:
                new.append(list(res))
        if not new:
            print("\n No new clauses. QUERY IS NOT ENTAILED.")
            return False
        clauses.extend(new)
kb = [['~human', 'x'], ('mortal', 'x')],
      [('human', 'socrates')]]
query = ('mortal', 'socrates')
resolution(kb, query)

```

Output:

```

===== RESTART: C:\Users\Jes
KNOWLEDGE BASE
Clause 1: [('~human', 'x'), ('mortal', 'x')]
Clause 2: [('human', 'socrates')]

Query: ('mortal', 'socrates')
Initial Clauses:
C1: [('~human', 'x'), ('mortal', 'x')]
C2: [('human', 'socrates')]
C3: [('~mortal', 'socrates')]

Step 1: Resolving [('~human', 'x'), ('mortal', 'x')] and [('human', 'socrates')]
=> (('mortal', 'socrates'),)
Step 2: Resolving [('~human', 'x'), ('mortal', 'x')] and [('~mortal', 'socrates')]
=> (('~human', 'socrates'),)
Step 3: Resolving [('~human', 'x'), ('mortal', 'x')] and [('human', 'socrates')]
=> (('mortal', 'socrates'),)
Step 4: Resolving [('~human', 'x'), ('mortal', 'x')] and [('~mortal', 'socrates')]
=> (('~human', 'socrates'),)
Step 5: Resolving [('human', 'socrates')] and [('~human', 'socrates')]
=> ()

Empty clause derived. QUERY IS ENTAILED.

```

Program 10

Implement Alpha-Beta Pruning.

```

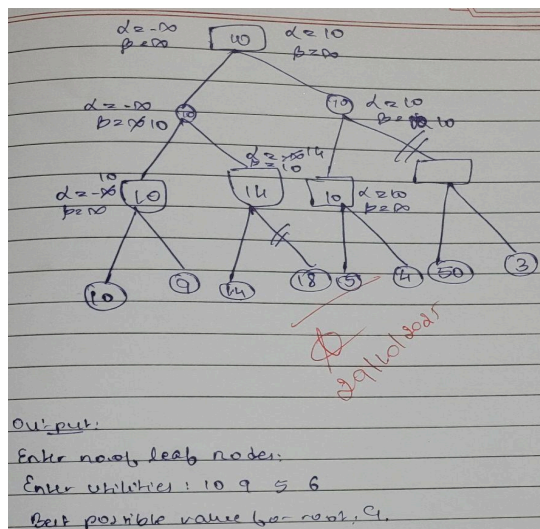
Algorithm:
alphabeta(node, depth, d, f, maxplayer)
{
    if (depth == 0 or isTerminalNode)
        return utility(node)
    if maxplayer == True:
        v = -∞
        for each child ← children(node):
            v = max(v, alphabeta(child, depth-1, d, f, False))
            if v >= d:
                break
        End for
    End if
    return v
}

```

```

else:
    v = ∞
    for each child ← children(node):
        v = min(v, alphabeta(child, depth-1, d, f, True))
        if v <= d:
            break
    End for
    return v
End

```



Code:

import math

def alpha_beta(index, depth, max_depth, alpha, beta, maximizingPlayer, values):

```

if depth == max_depth:
    return values[index]
if maximizingPlayer:
    v = float('-inf')
    for i in range(2):
        child_index = 2 * index + i
        v = max(v, alpha_beta(child_index, depth + 1, max_depth, alpha, beta, False, values))
        alpha = max(alpha, v)
        if beta <= alpha:
            break
    return v
else:
    v = float('inf')
    for i in range(2):
        child_index = 2 * index + i
        v = min(v, alpha_beta(child_index, depth + 1, max_depth, alpha, beta, True, values))
        beta = min(beta, v)
        if beta <= alpha:
            break
    return v
n = int(input("Enter number of leaf nodes (power of 2): "))
values = []
for i in range(n):
    val = int(input(f"Enter utility value of leaf {i+1}: "))
    values.append(val)
depth = int(math.log2(n))
tree = []
index = 0
for d in range(depth + 1):
    level_nodes = 2 ** d
    level = []
    for i in range(level_nodes):
        if d == depth:
            level.append(values[index])
            index += 1
        else:
            level.append(None)
    tree.append(level)
for d in range(depth - 1, -1, -1):
    for i in range(len(tree[d])):
        tree[d][i] = '?'
print("\nGame Tree (top = root):")
for level in tree:
    print(level)
best_value = alpha_beta(0, 0, depth, float('-inf'), float('inf'), True, values)
print("\nBest possible value for MAX at root:", best_value)

```

Output:

```
===== RESTART: C:\Users\Jesvita\OneDrive\  
Enter number of leaf nodes (power of 2): 4  
Enter utility value of leaf 1: 6  
Enter utility value of leaf 2: 7  
Enter utility value of leaf 3: -8  
Enter utility value of leaf 4: 0  
  
Game Tree (top = root):  
['?']  
['?', '?']  
[6, 7, -8, 0]  
  
Best possible value for MAX at root: 6  
|
```